ASSIGNMENT-8.2

ROLL NO:2303A51360

BATCH:29

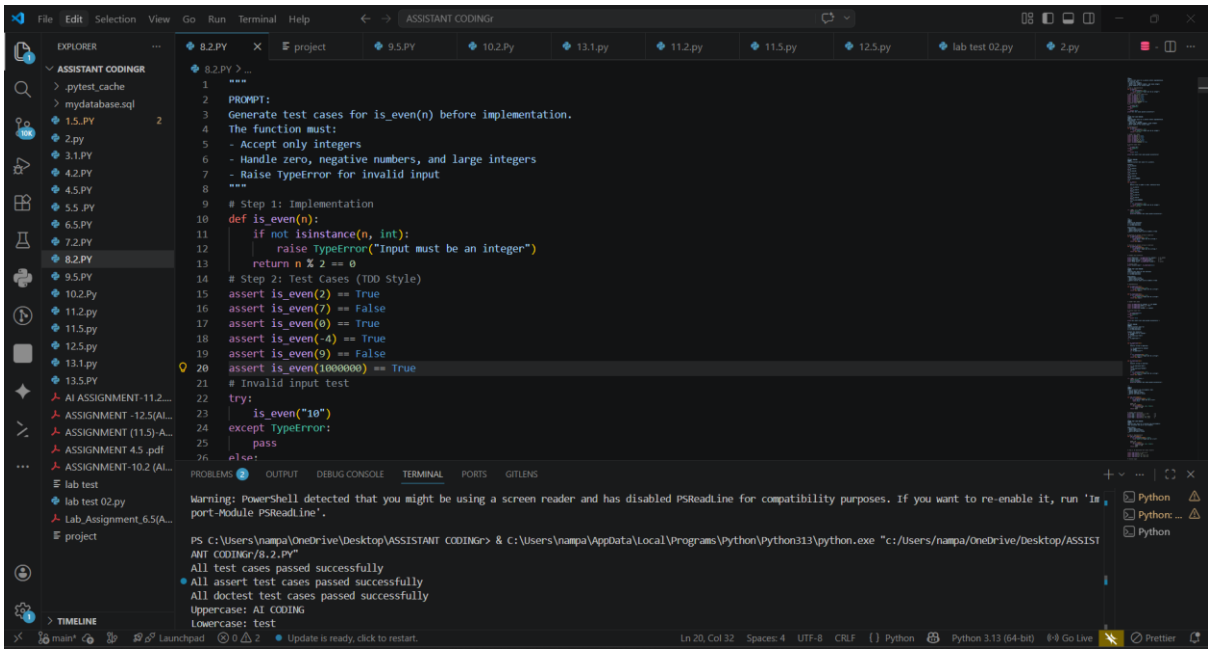
TASK-1:

PROMPT:

Generate test cases for a Python function is_even(n) before implementation.The function must:

- Accept only integers
- Handle zero, negative numbers, and large integers
- Raise TypeError for invalid input

CODE:



OBSERVATION:

- The function correctly identifies even and odd numbers.
- It handles zero, negative numbers, and large integers properly.
- Invalid input such as string raises TypeError.

- This version verifies functionality through manual execution and printed output.
- It confirms logical correctness but does not automate testing fully.

2. ASSERY TEST CASE VERSION:

PROMPT :

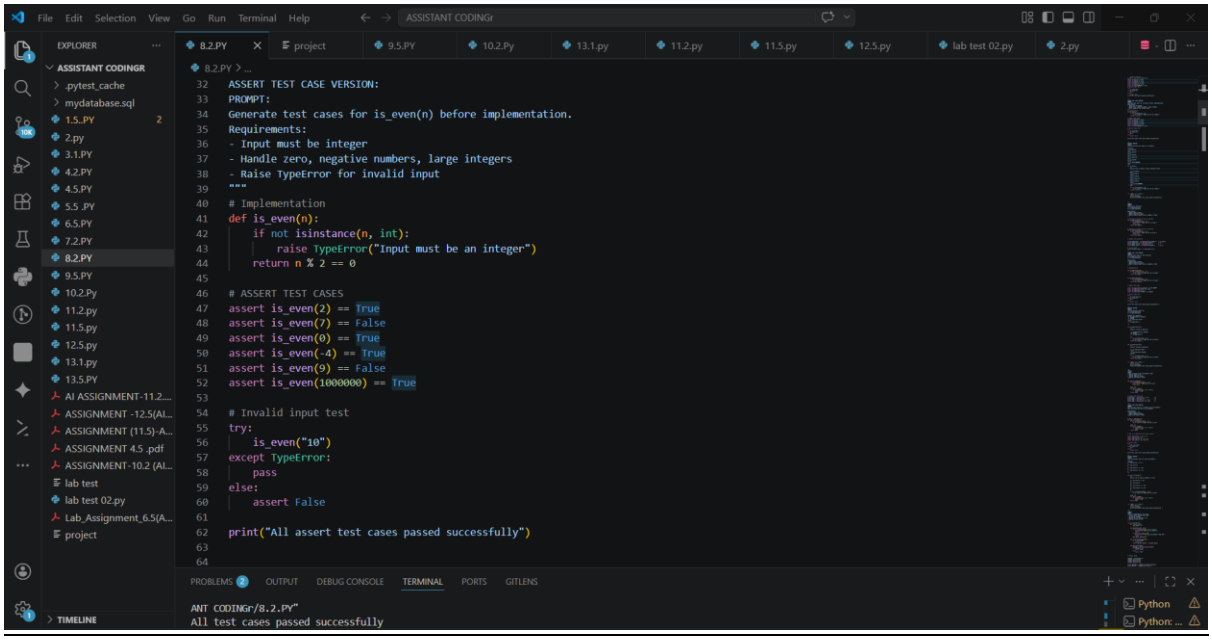
Generate assert-based test cases for is_even(n) following TDD approach.
Requirements:

Input must be integer

Handle zero, negative numbers, and large integers

Raise TypeError for invalid input

CODE:



```
ASSERT TEST CASE VERSION:
PROMPT:
Generate test cases for is_even(n) before implementation.
Requirements:
- Input must be integer
- Handle zero, negative numbers, large integers
- Raise TypeError for invalid input
"""
# Implementation
def is_even(n):
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")
    return n % 2 == 0

# ASSERT TEST CASES
assert is_even(2) == True
assert is_even(7) == False
assert is_even(0) == True
assert is_even(-4) == True
assert is_even(9) == False
assert is_even(1000000) == True

# Invalid input test
try:
    is_even("10")
except TypeError:
    pass
else:
    assert False

print("All assert test cases passed successfully")
```

```
ANT CODINGr/8.2.PY"
All test cases passed successfully
```

OBSERVATION:

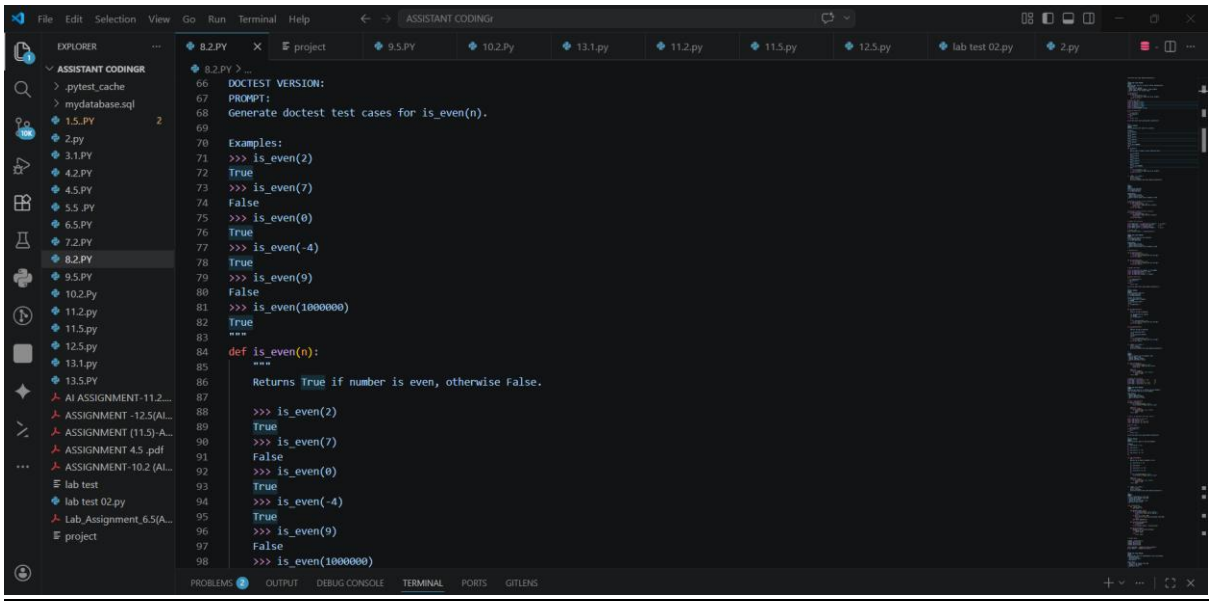
- All assertions passed successfully.
- Automated validation ensures correctness.
- Exception handling was verified using try-except.
- Assert-based testing is faster and more reliable than manual checking.

3. DOCTEST VERSION:

PROMPT:

Generate doctest test cases inside the function documentation for is_even(n).

CODE:



```
DOCTEST VERSION:
PROMPT:
Generate doctest test cases for is_even(n).

Examples:
>>> is_even(2)
True
>>> is_even(7)
False
>>> is_even(0)
True
>>> is_even(-4)
True
>>> is_even(9)
False
>>> is_even(1000000)
True
"""
def is_even(n):
    """
    Returns True if number is even, otherwise False.

    >>> is_even(2)
    True
    >>> is_even(7)
    False
    >>> is_even(0)
    True
    >>> is_even(-4)
    True
    >>> is_even(9)
    False
    >>> is_even(1000000)
```

OBSERVATION:

Doctest integrates testing inside the function's docstring.

Examples serve as both documentation and test validation.

Running doctest.testmod() confirms correctness automatically.

This version improves readability and documentation quality.

Suitable for small utility functions.

TASK - 2 :

PROMPT:

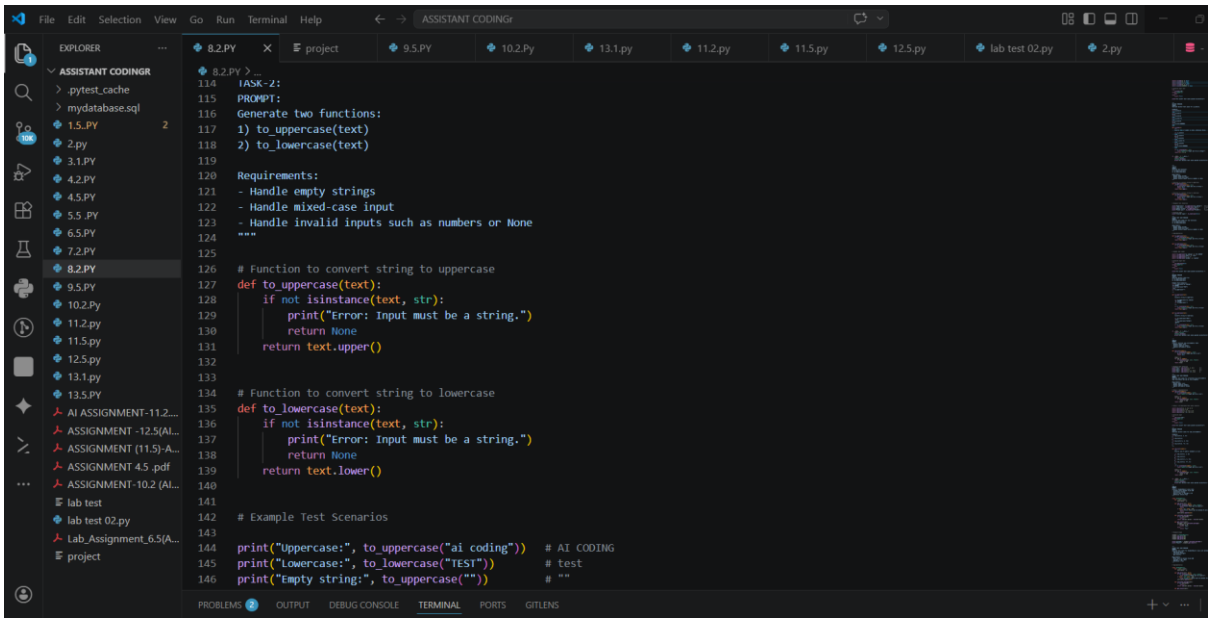
Generate two functions:

1. to_uppercase(text)
2. to_lowercase(text)

Requirements:

- Handle empty strings
- Handle mixed-case input
- Handle invalid inputs such as numbers or None

CODE:



```
TASK-2:
PROMPT:
Generate two functions:
1) to_uppercase(text)
2) to_lowercase(text)

Requirements:
- Handle empty strings
- Handle mixed-case input
- Handle invalid inputs such as numbers or None
"""

# Function to convert string to uppercase
def to_uppercase(text):
    if not isinstance(text, str):
        print("Error: Input must be a string.")
        return None
    return text.upper()


# Function to convert string to lowercase
def to_lowercase(text):
    if not isinstance(text, str):
        print("Error: Input must be a string.")
        return None
    return text.lower()


# Example Test Scenarios

print("Uppercase:", to_uppercase("ai coding"))   # AI CODING
print("Lowercase:", to_lowercase("TEST"))        # test
print("Empty string:", to_uppercase(""))         # ""
```

OBSERVATION:

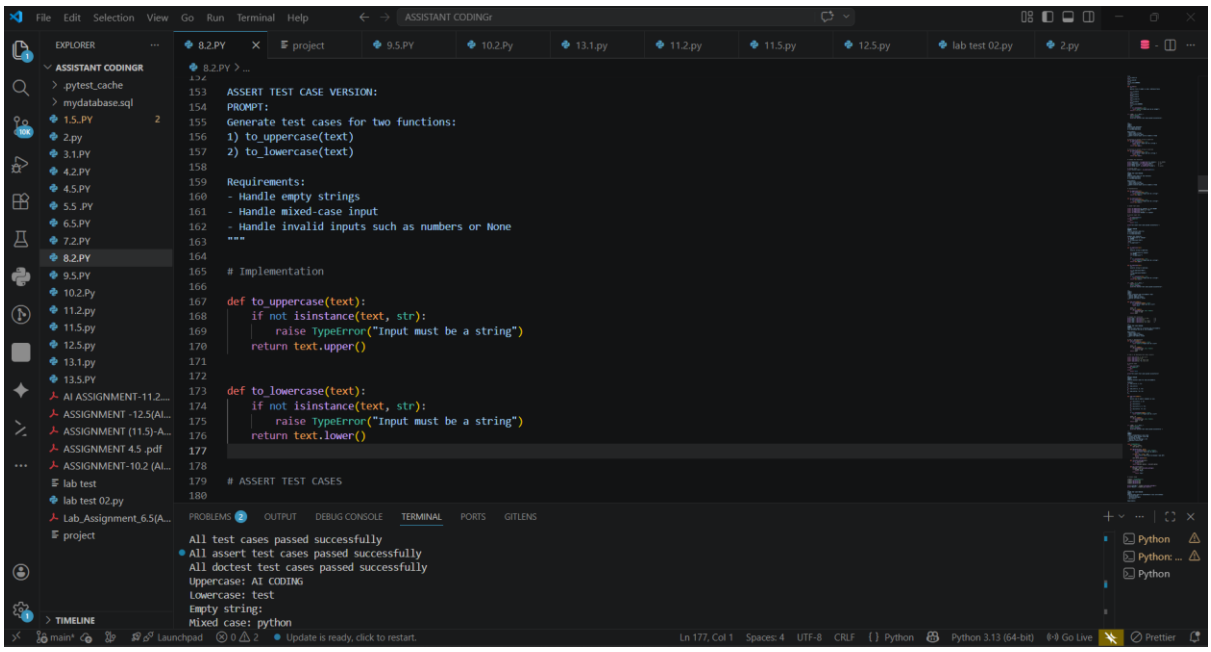
- Functions correctly convert strings to upper and lower case.
- Mixed-case and empty string scenarios are handled properly.
- Invalid inputs display error message and return None.
- Basic functionality verified through print statements.

2. ASSERY TEST CASE VERSION:

PROMPT :

Generate assert test cases for to_uppercase() and to_lowercase() with proper error handling.

CODE:



OBSERVATION:

_All assertions passed successfully.

TypeError is raised for invalid input.

Automated validation ensures strong correctness.

More robust than print-based version.

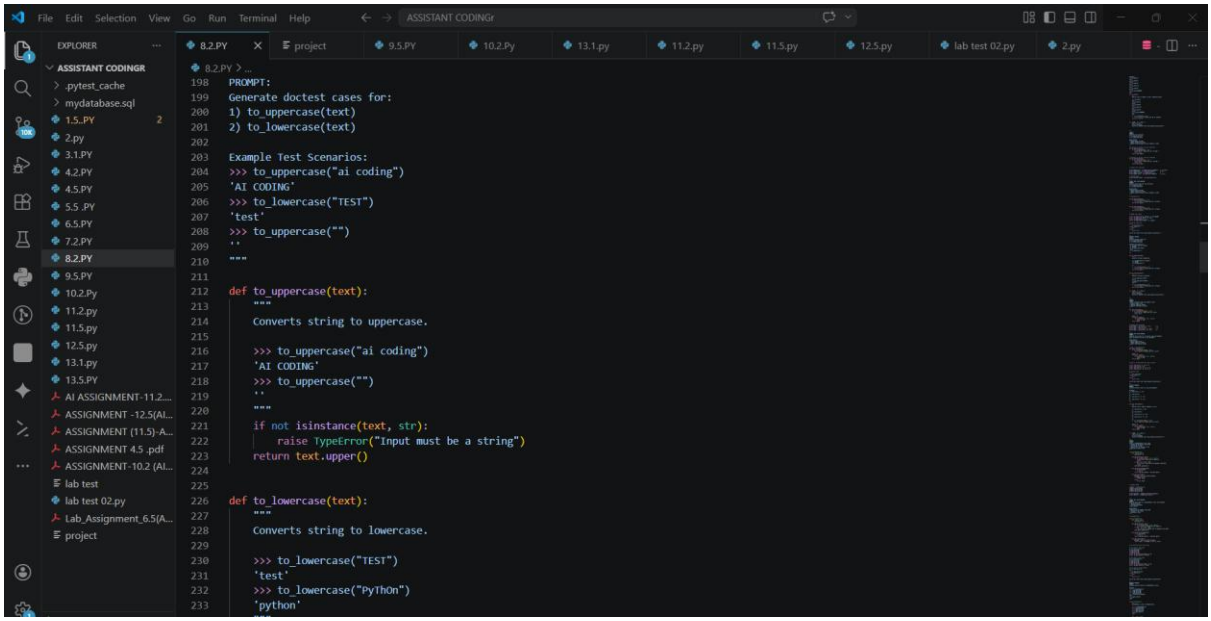
Proper exception handling improves reliability.

3. DOCTEST VERSION:

PROMPT:

Generate doctest examples for both string conversion functions.

CODE:



```
PROMPT:
Generate doctest cases for:
1) to_uppercase(text)
2) to_lowercase(text)

Example Test Scenarios:
>>> to_uppercase("ai coding")
'AI CODING'
>>> to_lowercase("TEST")
'test'
>>> to_uppercase("")
''
"""

def to_uppercase(text):
    """
    Converts string to uppercase.

    >>> to_uppercase("ai coding")
    'AI CODING'
    >>> to_uppercase("")
    ''
    """
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.upper()


def to_lowercase(text):
    """
    Converts string to lowercase.

    >>> to_lowercase("TEST")
    'test'
    >>> to_lowercase("PyThOn")
    'python'
```

OBSERVATION:

Doctest examples clearly show expected input and output.

Tests double as documentation.

Function behavior is automatically verified.

Enhances clarity and professional code documentation.

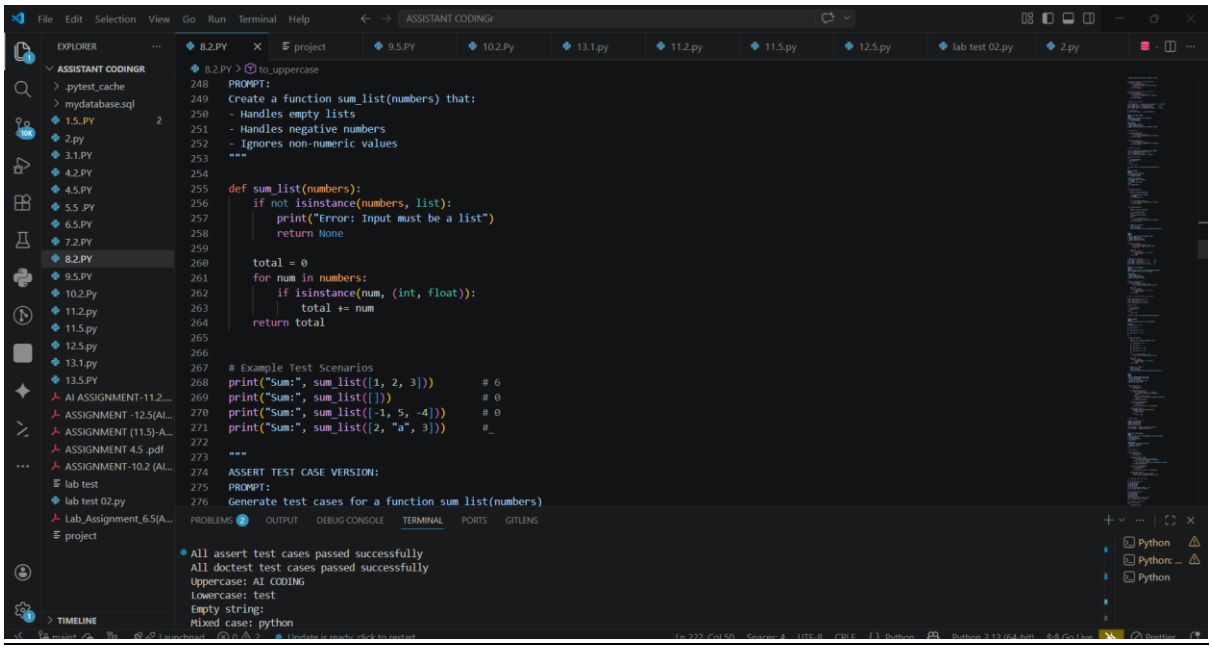
TASK:3

PROMPT:

Create a function sum_list(numbers) that:

- Handles empty lists
- Handles negative numbers
- Ignores non-numeric values

CODE:



```
PROMPT:
Create a function sum_list(numbers) that:
- Handles empty lists
- Handles negative numbers
- Ignores non-numeric values
"""

def sum_list(numbers):
    if not isinstance(numbers, list):
        print("Error: Input must be a list")
        return None

    total = 0
    for num in numbers:
        if isinstance(num, (int, float)):
            total += num
    return total


# Example Test Scenarios
print("Sum:", sum_list([1, 2, 3]))        # 6
print("Sum:", sum_list([]))               # 0
print("Sum:", sum_list([-1, 5, -4]))      # 0
print("Sum:", sum_list([2, "a", 3]))      #_

"""
ASSERT TEST CASE VERSION:
PROMPT:
Generate test cases for a function sum list(numbers)
```

```
All assert test cases passed successfully
All doctest test cases passed successfully
Uppercase: AI CODING
Lowercase: test
Empty string:
Mixed case: python
```

OBSERVATION:

_Function successfully sums numeric elements.

Returns 0 for empty list.

Ignores non-numeric values safely.

Prints error for invalid input type.

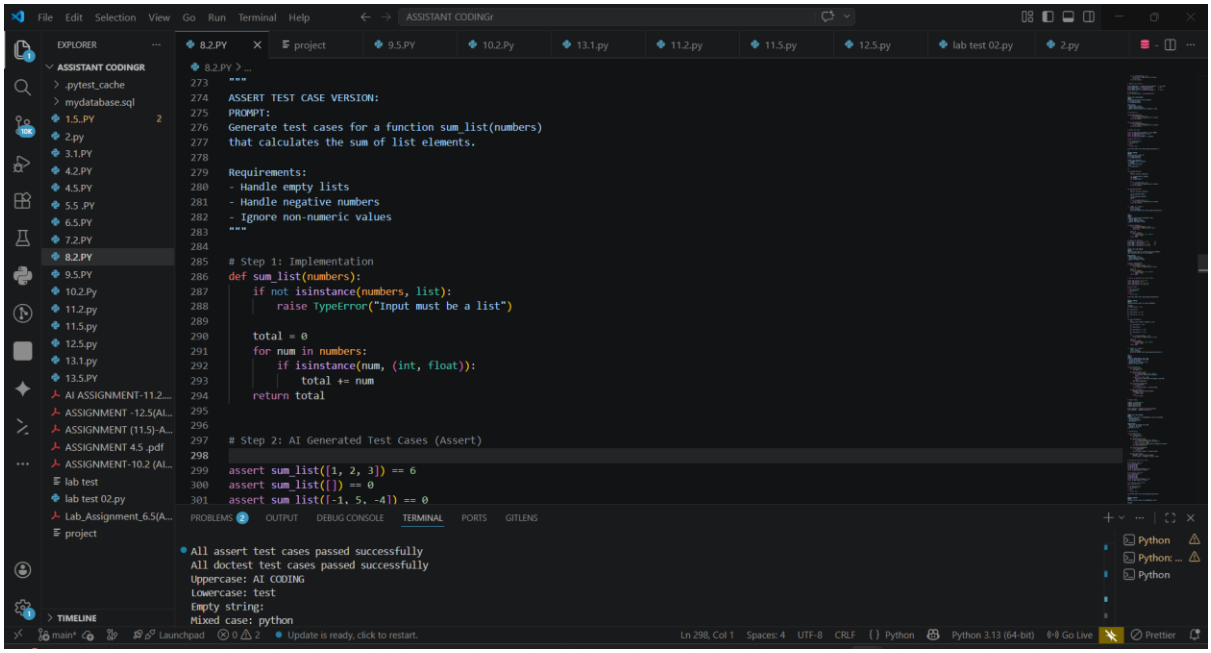
Works correctly but lacks automated verification.

3. ASSERT TEST CASE VERSION:

PROMPT :

Generate assert test cases for sum_list(numbers) based on given requirements.

CODE:



OBSERVATION:

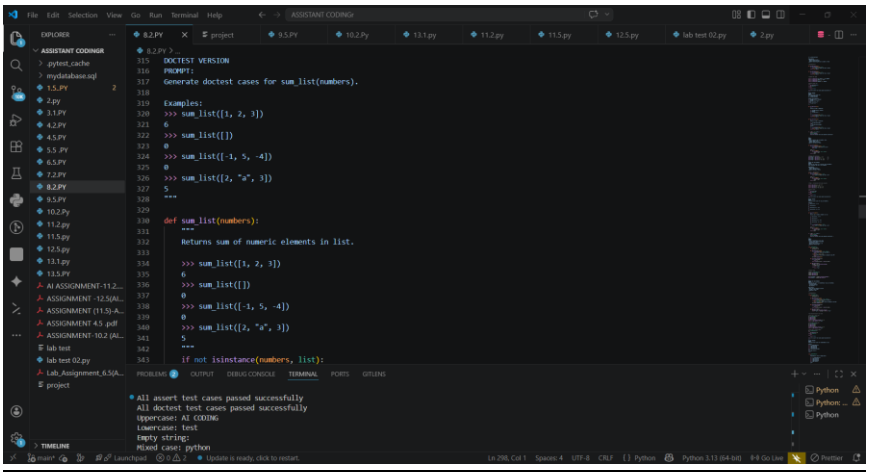
- All test cases passed.
- Proper TypeError raised for invalid input.
- Automated testing validates edge cases.
- Ensures robustness and correctness.
- Demonstrates effective TDD approach.

3. DOCTEST VERSION:

PROMPT:

Generate doctest cases for sum_list(numbers) including edge cases

CODE:



:

OBSERVATION:

Examples are embedded in documentation.

Running doctest validates correctness.

Improves readability and maintainability.

Suitable for lightweight function testing.

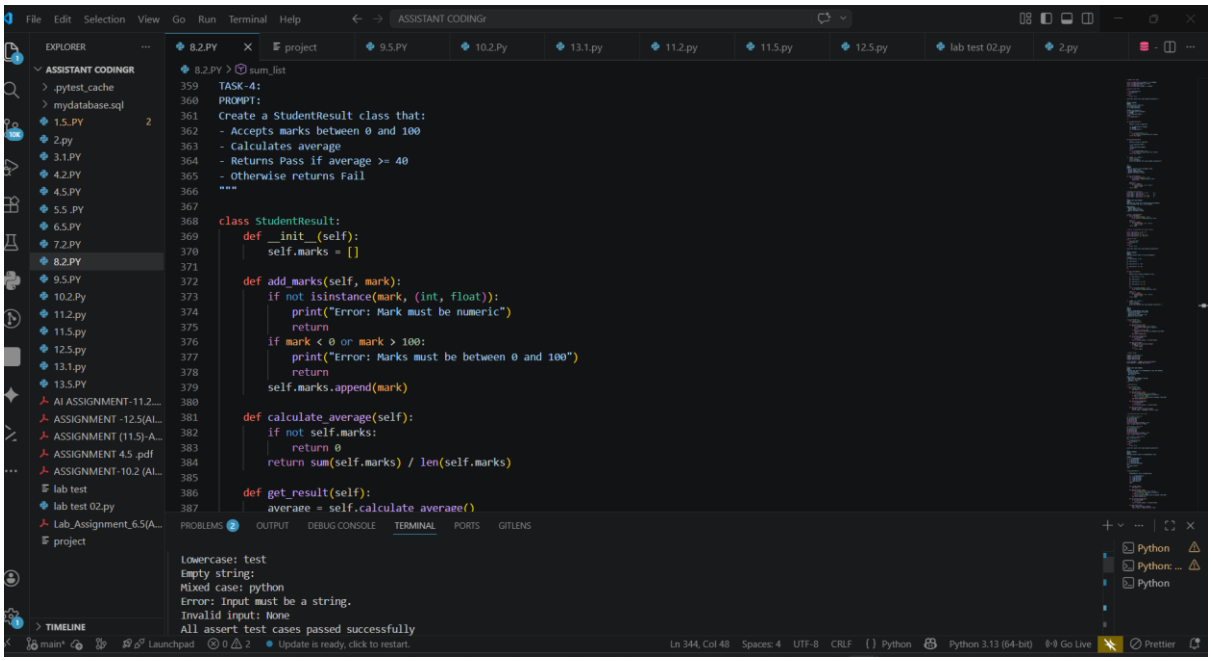
TASK-4:

PROMPT:

Create a StudentResult class that:

- Accepts marks between 0 and 100
- Calculates average
- Returns Pass if average ≥ 40
- Otherwise returns Fail

CODE:



```
TASK-4:
PROMPT:
Create a StudentResult class that:
- Accepts marks between 0 and 100
- Calculates average
- Returns Pass if average >= 40
- Otherwise returns Fail
"""

class StudentResult:
    def __init__(self):
        self.marks = []

    def add_marks(self, mark):
        if not isinstance(mark, (int, float)):
            print("Error: Mark must be numeric")
            return
        if mark < 0 or mark > 100:
            print("Error: Marks must be between 0 and 100")
            return
        self.marks.append(mark)

    def calculate_average(self):
        if not self.marks:
            return 0
        return sum(self.marks) / len(self.marks)

    def get_result(self):
        average = self.calculate_average()
```

```
Lowercase: test
Empty string:
Mixed case: python
Error: Input must be a string.
Invalid input: None
All assert test cases passed successfully
```

OBSERVATION:

Class stores marks correctly.

Calculates average accurately.

Returns correct result (Pass/Fail).

Invalid marks print error message.

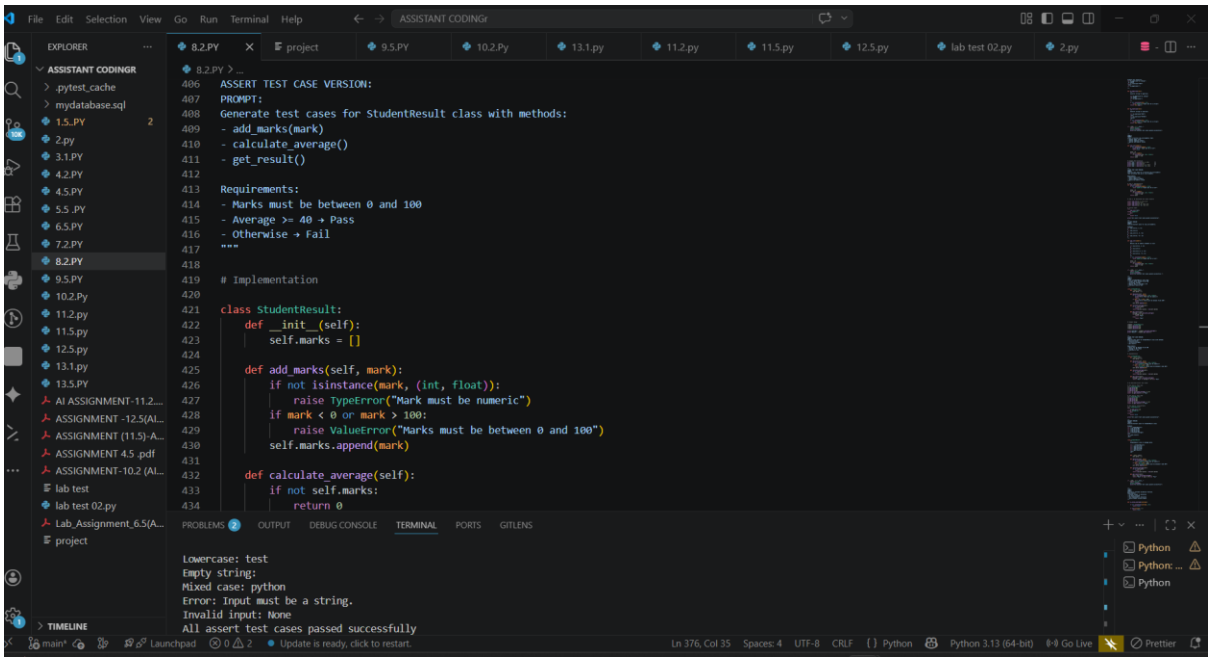
Basic testing done through manual execution.

2. ASSERY TEST CASE VERSION:

PROMPT :

Generate assert-based test cases for StudentResult class following TDD.

CODE:



```
ASSERT TEST CASE VERSION:
PROMPT:
Generate test cases for StudentResult class with methods:
- add_marks(mark)
- calculate_average()
- get_result()

Requirements:
- Marks must be between 0 and 100
- Average >= 40 → Pass
- Otherwise → Fail
"""

# Implementation

class StudentResult:
    def __init__(self):
        self.marks = []

    def add_marks(self, mark):
        if not isinstance(mark, (int, float)):
            raise TypeError("Mark must be numeric")
        if mark < 0 or mark > 100:
            raise ValueError("Marks must be between 0 and 100")
        self.marks.append(mark)

    def calculate_average(self):
        if not self.marks:
            return 0
```

```
Lowercase: test
Empty string:
Mixed case: python
Error: Input must be a string.
Invalid input: None
All assert test cases passed successfully
```

OBSERVATION:

Pass and Fail cases validated successfully.

ValueError raised for invalid marks.

Automated testing improves reliability.

Better validation compared to print-based version.

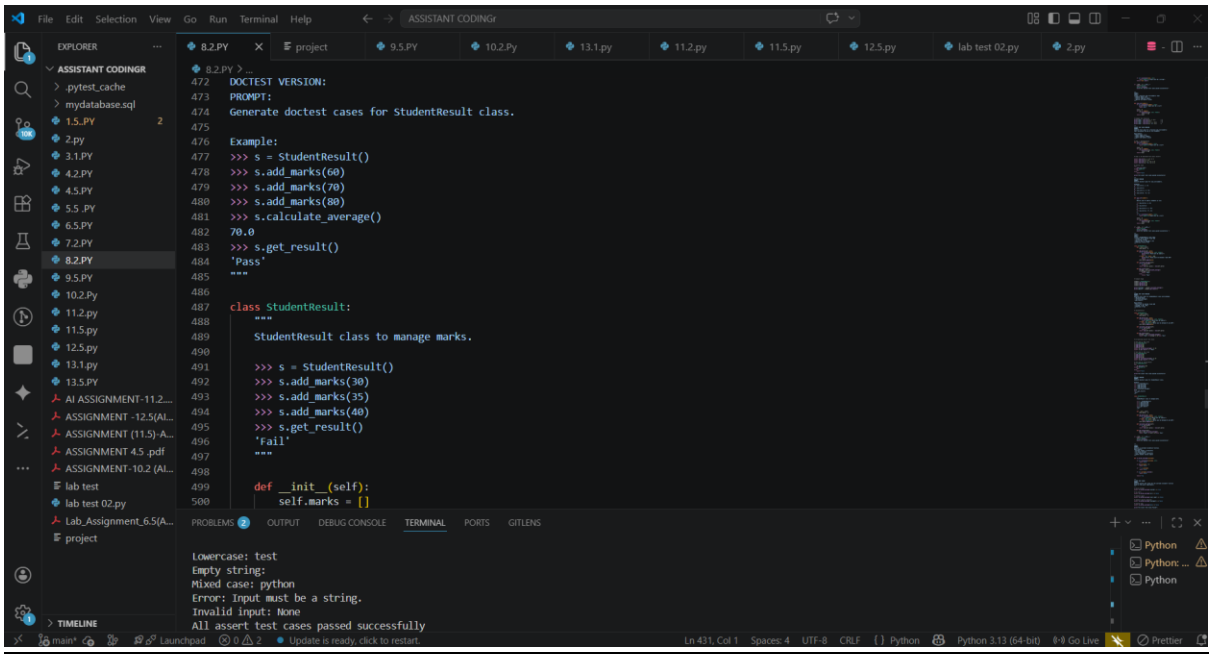
Demonstrates strong test-driven design.

3. DOCTEST VERSION:

PROMPT:

Generate doctest cases for StudentResult class methods.

CODE:



OBSERVATION:

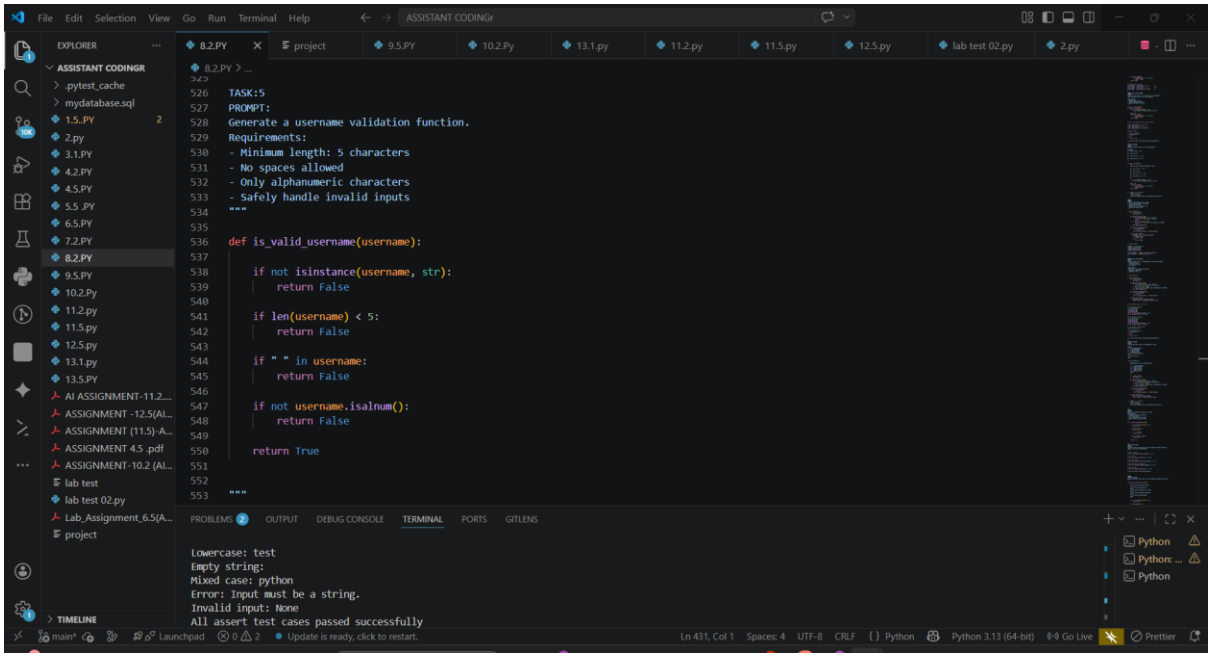
- Class usage documented through examples.
- Doctest validates behavior automatically.
- Enhances documentation quality.
- Suitable for academic demonstration of TDD.

TASK:5

PROMPT:

Generate a username validation function with:

CODE:



OBSERVATION:

Function validates length and format correctly.

Returns False for invalid inputs.

Works correctly for required scenarios.

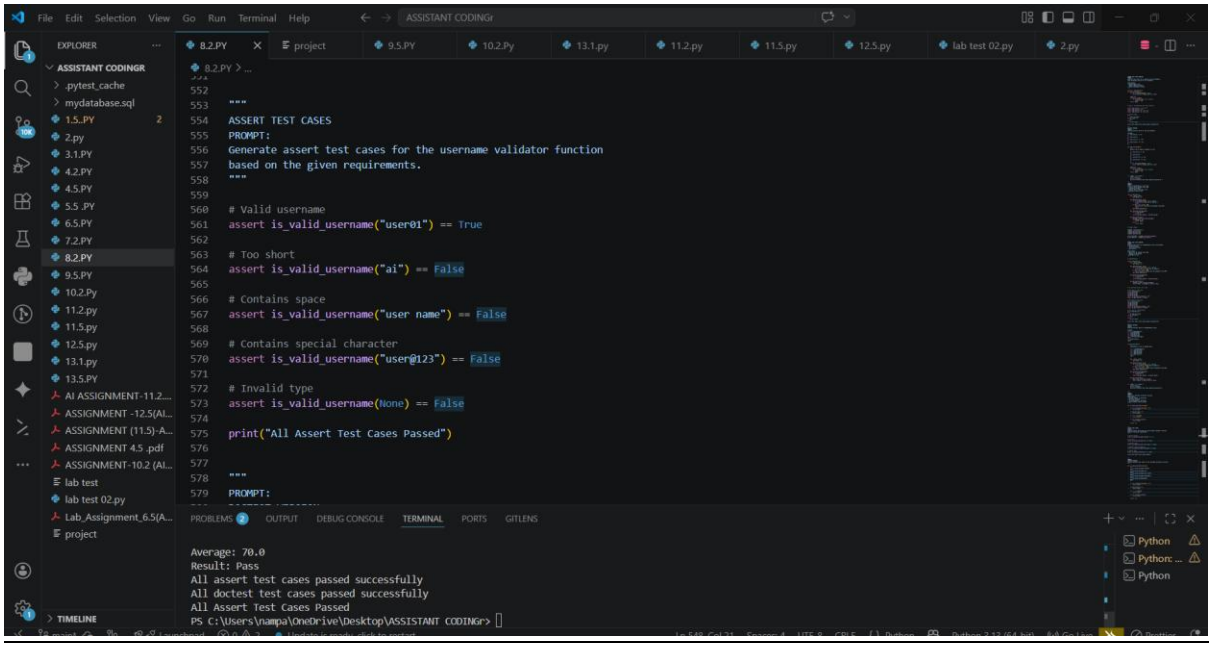
Basic validation tested manually.

2. ASSERY TEST CASE VERSION:

PROMPT :

Generate assert test cases for username validator.

CODE:



OBSERVATION:

All assertions passed successfully.

Valid and invalid cases covered.

Automated testing ensures correctness.

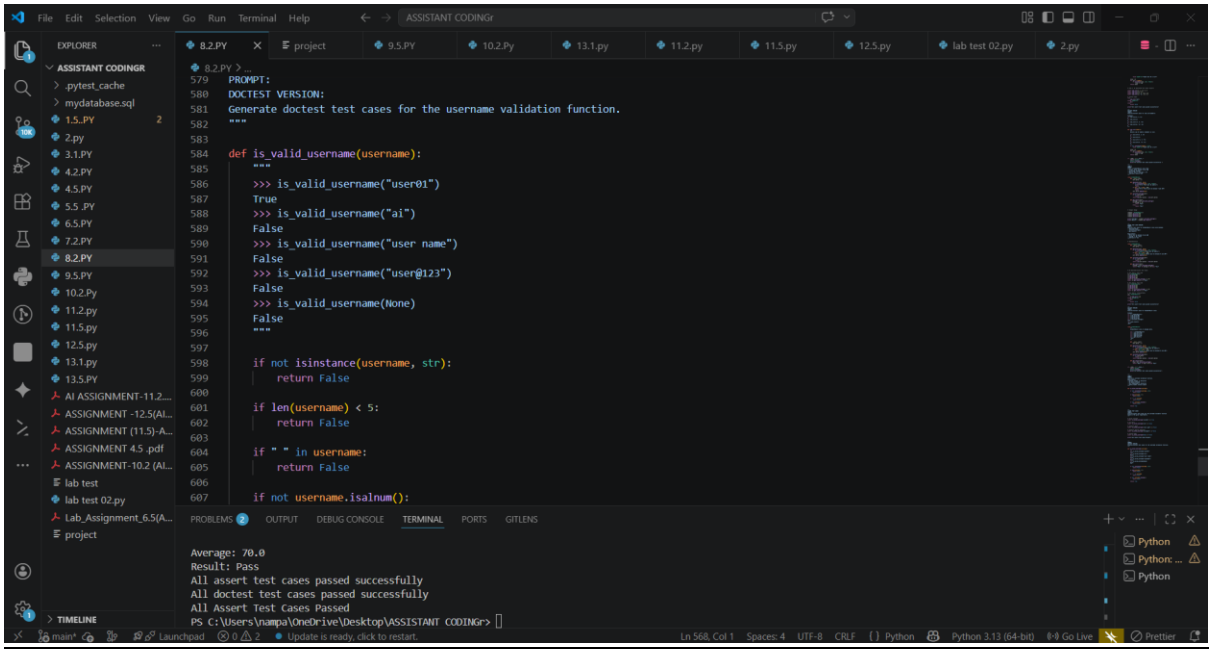
Proper validation of edge cases.

3. DOCTEST VERSION:

PROMPT:

Generate doctest examples for username validation function.

CODE:

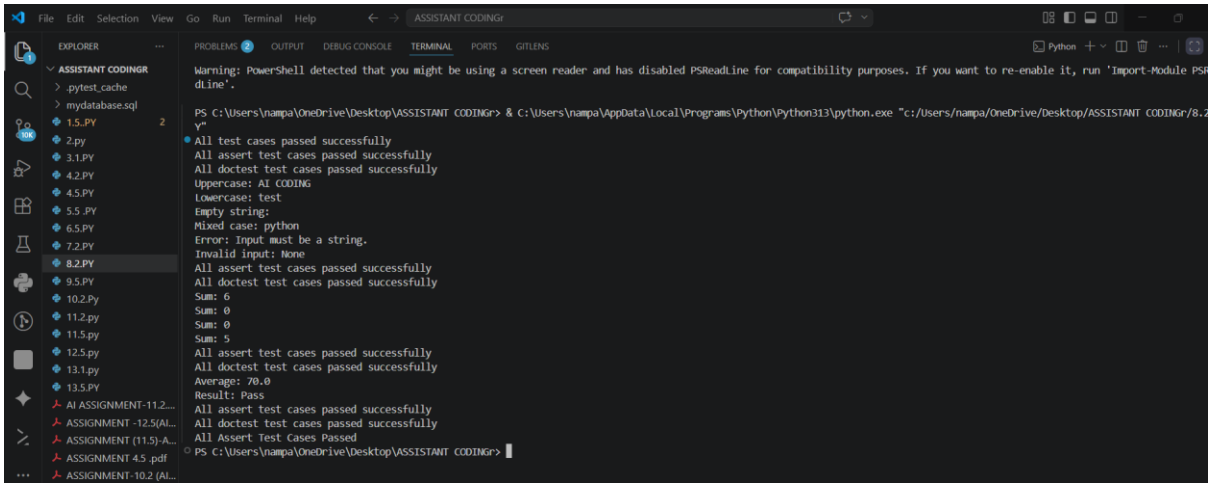


OBSERVATON:

Examples clearly describe expected behavior.

Doctest execution confirms correctness.

Improves documentation and code clarity.

Suitable for small validation functions.

ALL OUTPUTS: